

Inverse Problems in Geophysics

Part 10: Nonlinear minimization

2. MGPY+MGIN

Thomas Günther

thomas.guenther@geophysik.tu-freiberg.de

Recap

- non-linear problems linearized: $\mathbf{S}\Delta\mathbf{m} = \Delta\mathbf{d}$
- sensitivity: derivative of forward response $\partial f_i / \partial m_j$
- Jacobian: sensitivity with model & data transforms (inner derivatives)
- objective function $\Phi = \Phi_d + \lambda\Phi_m$ (data + model)
- model update in direction of negative gradient $\Rightarrow$ Steepest Descent

A simple problem

Formula for 2 model parameters m_1/m_2 & data measured at x

$$f(m) = \sin(\omega m_1 x_i) + m_1 m_2$$

Derivatives:

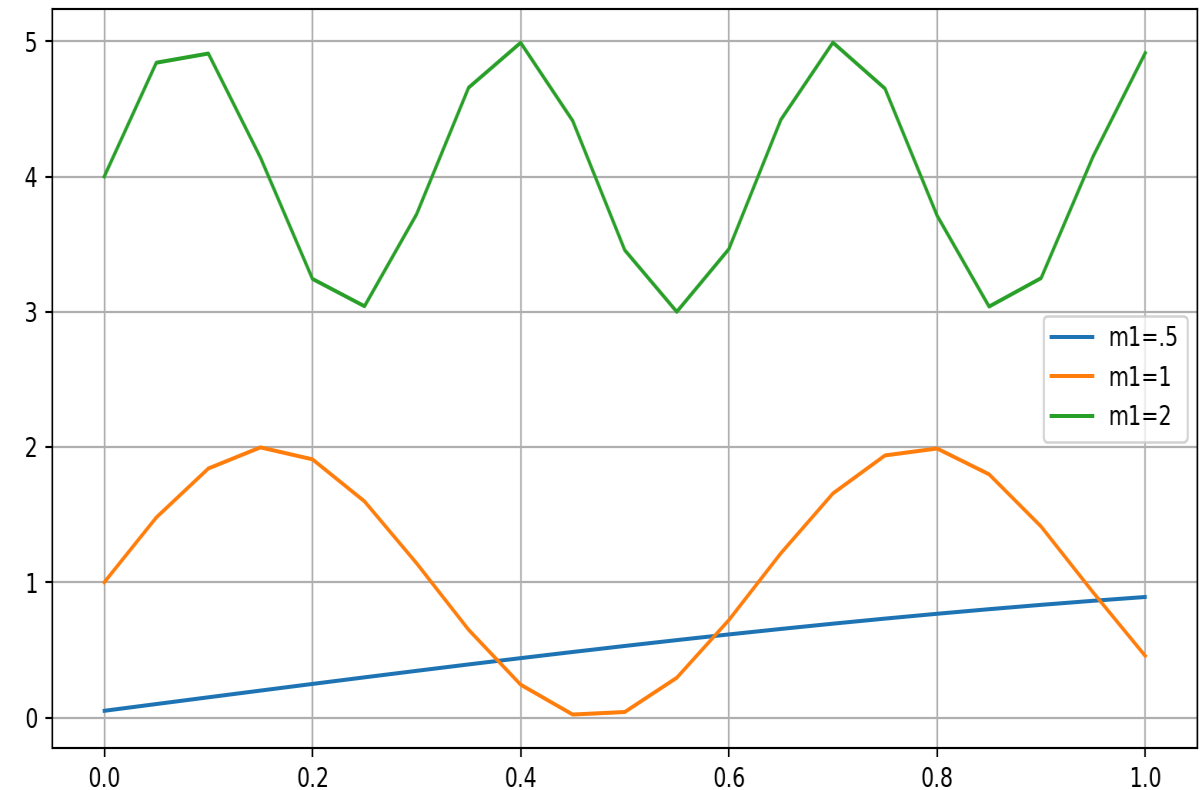
$$\frac{\partial f(m)}{\partial m_1} = \omega x_i \cos(\omega m_1 x_i) + m_2$$

$$\frac{\partial f(m)}{\partial m_2} = m_1$$

A simple minimization problem

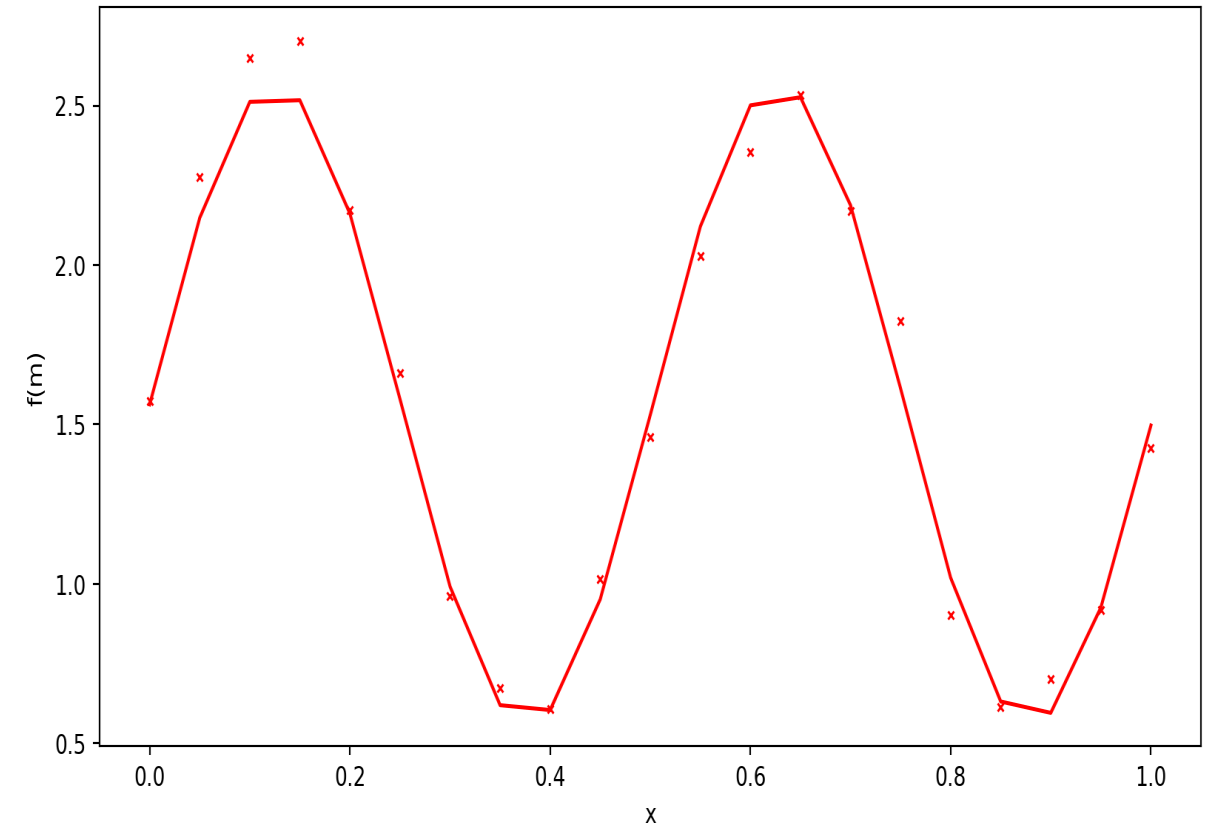
$$f(m) = \sin(\omega m_1 x_i) + m_1 m_2$$

```
1 x = np.arange(0, 1.001, 0.05)
2 om = 10
3 forward = lambda m: np.sin(om*x*m[0])+m[0]*m[1]
4 plt.plot(x, forward([0.1, 0.5]), label="m1=.5")
5 plt.plot(x, forward([1, 1]), label="m1=1")
6 plt.plot(x, forward([2, 2]), label="m1=2");
7 plt.grid()
8 plt.legend();
```



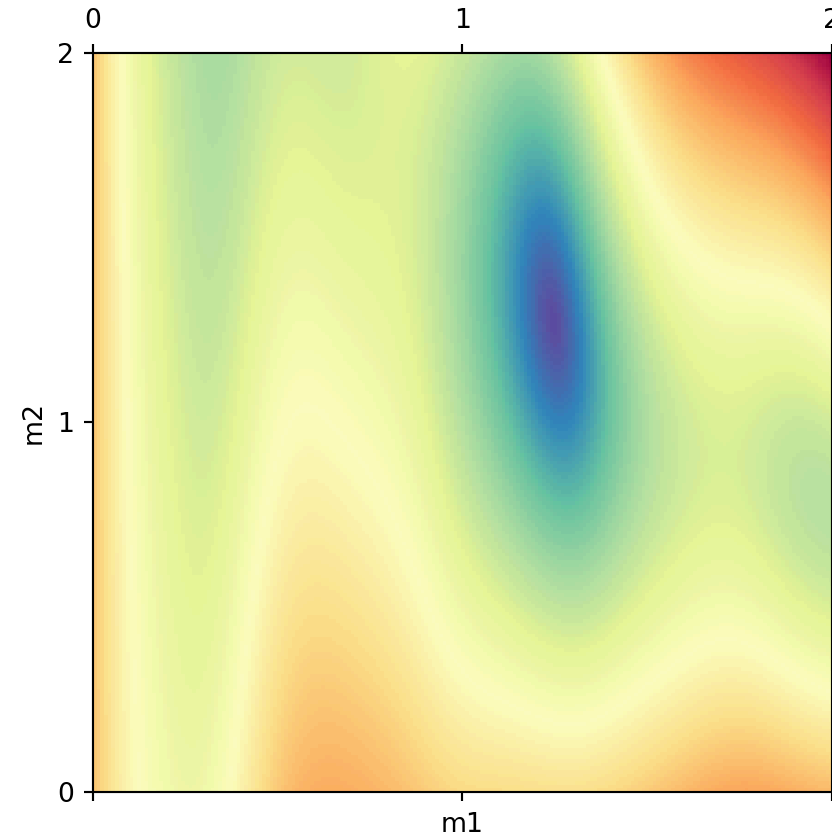
Synthetic data

```
1 mSynth = [1.25, 1.25]
2 data = forward(mSynth);
3 plt.plot(x, data, "r-")
4 data += np.random.randn(len(data))*0.1;
5 plt.plot(x, data, "rx");
6 plt.xlabel("x")
7 plt.ylabel("f(m)");
```



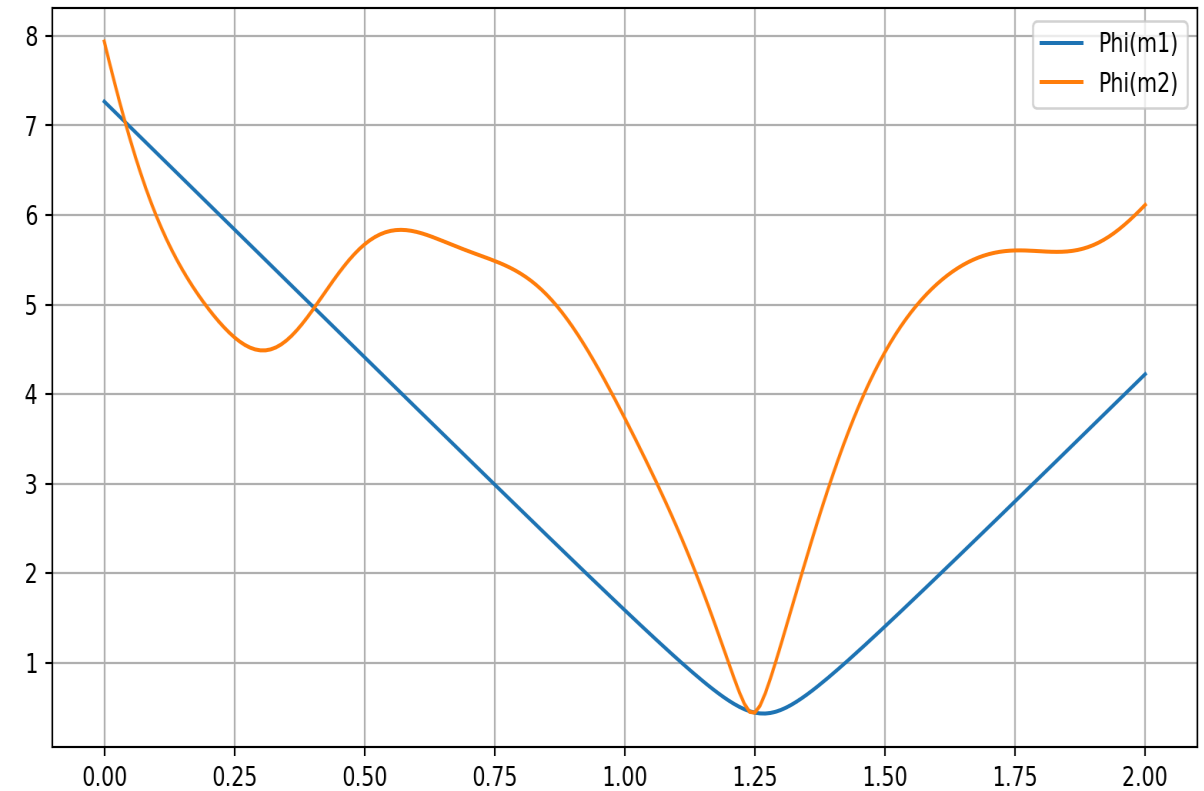
Objective function

```
1 ms = np.arange(0, 2.001, 0.01)
2 Phi = np.zeros([len(ms), len(ms)])
3 for i, m1 in enumerate(ms):
4     for j, m2 in enumerate(ms):
5         Phi[i, j] = np.linalg.norm(
6             data-forward([m1, m2]))
7
8 fig, ax = plt.subplots()
9 ax.matshow(Phi.T, cmap="Spectral_r",
10            extent=ms[[0, -1, -1, 0]])
11 ax.set_xlabel("m1")
12 ax.set_ylabel("m2")
13 ax.invert_yaxis();
```



Objective function

```
1 plt.plot(ms, Phi[int(mSynth[0]*100), :],  
2          label="Phi(m1)")  
3 plt.plot(ms, Phi[:, int(mSynth[1]*100)],  
4          label="Phi(m2)")  
5 plt.grid()  
6 plt.legend();
```



Gradient methods

$$\Phi_d = (\mathbf{d} - \mathbf{f}(\mathbf{m}))^T (\mathbf{d} - \mathbf{f}(\mathbf{m}))$$

$$\mathbf{b} = \nabla \Phi_d = -2\mathbf{S}^T (\mathbf{d} - \mathbf{f}(\mathbf{m})) \quad \text{with} \quad S_{ij} = \frac{\partial f_i(\mathbf{m}^0)}{\partial m_j}$$

Steepest descent method: $\Delta \mathbf{m} = \nu = -\mathbf{b}/|\mathbf{b}|$

Regularization: $\Phi = \Phi_d + \lambda \Phi_m = \Phi_d + \lambda (\mathbf{c} - \mathbf{C}\mathbf{m})^T (\mathbf{c} - \mathbf{C}\mathbf{m})$

$$\mathbf{b} = \nabla \Phi = -2\mathbf{S}^T (\mathbf{d} - \mathbf{f}(\mathbf{m})) - 2\mathbf{C}^T (\mathbf{c} - \mathbf{C}\mathbf{m})$$

Line search using Armijo rule

$$\mathbf{m}^{k+1} = \mathbf{m}^k + \alpha \nu$$

① Armijo's rule to determine α

$$\Phi_d(\mathbf{m}^{k+1}) \leq \Phi_d(\mathbf{m}^k) + c\alpha \nu^T \nabla \Phi_d$$

- c -empirical constant, typically 1e-4
- start with large α and decrease until rule is fulfilled

⇒ Notebook on simple model

Gauss-Newton method

$$B_{ij} = \frac{\partial^2 \Phi_d}{\partial m_i \partial m_j} = \frac{\partial b_i}{\partial m_j} \approx 2\mathbf{S}^T \mathbf{S}$$

$$\Rightarrow \mathbf{m} - \mathbf{m}^0 = -\mathbf{B}^{-1} \mathbf{b} = (\mathbf{S}^T \mathbf{S})^{-1} \mathbf{S}^T (\mathbf{d} - \mathbf{f}(\mathbf{m}))$$

$\Rightarrow$ least-squares solution for $\Delta \mathbf{m} / \Delta \mathbf{d}$

with regularization term $\mathbf{B} = 2\mathbf{S}^T \mathbf{S} + 2\lambda \mathbf{C}^T \mathbf{C}$

$$\Rightarrow \Delta \mathbf{m} = (\mathbf{S}^T \mathbf{S} + \lambda \mathbf{C}^T \mathbf{C})^{-1} (\mathbf{S}^T (\mathbf{d} - \mathbf{f}(\mathbf{m})) + \lambda \mathbf{C}^T (\mathbf{c} - \mathbf{Cm}))$$

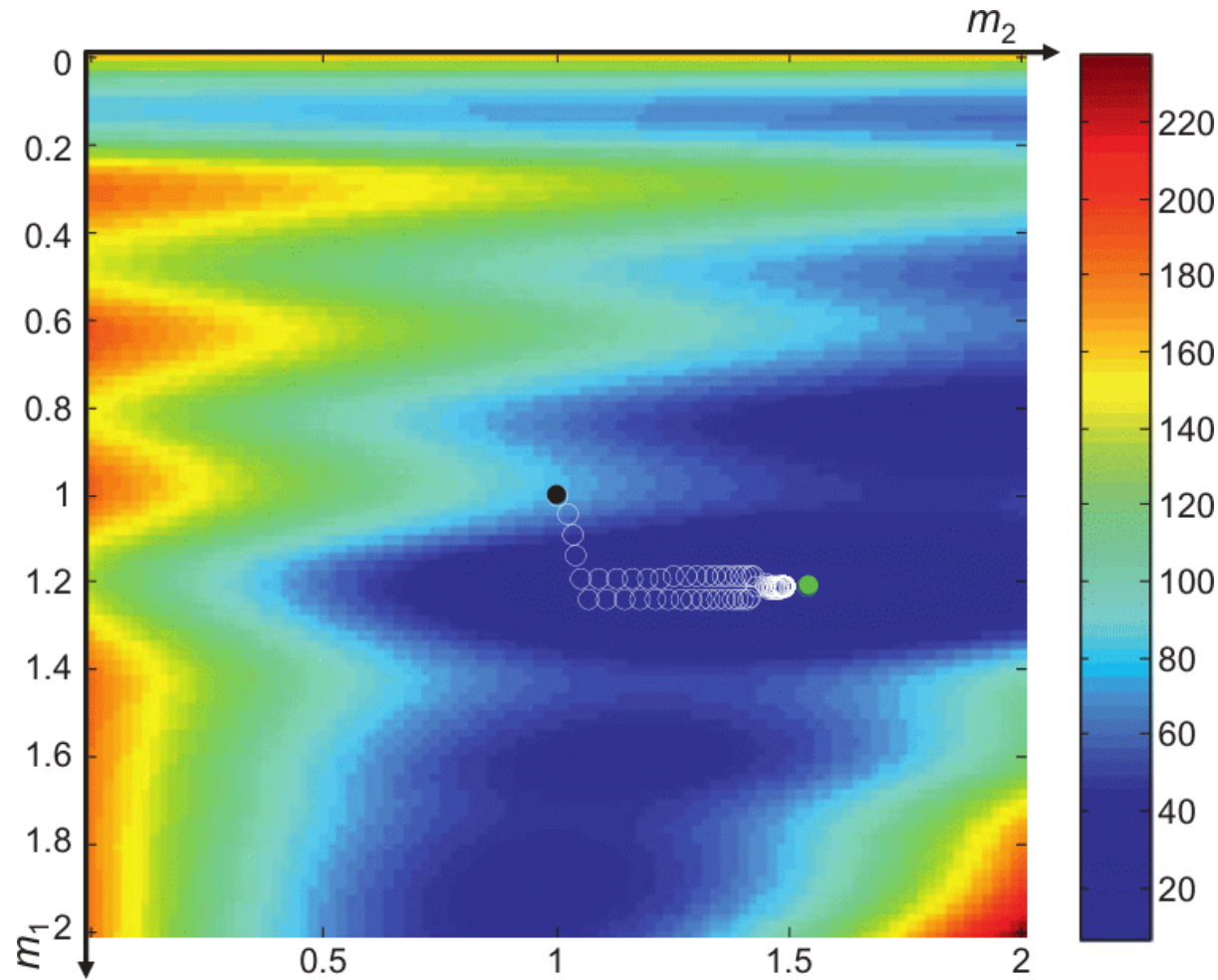
Gauss-Newton minimization

1. Choose starting model $\mathbf{m}^0$ ($n=0$) & compute model response $\mathbf{f}(\mathbf{m}^n)$
2. Compute sensitivity matrix $\mathbf{S}^n$
3. Solve linearized subproblem

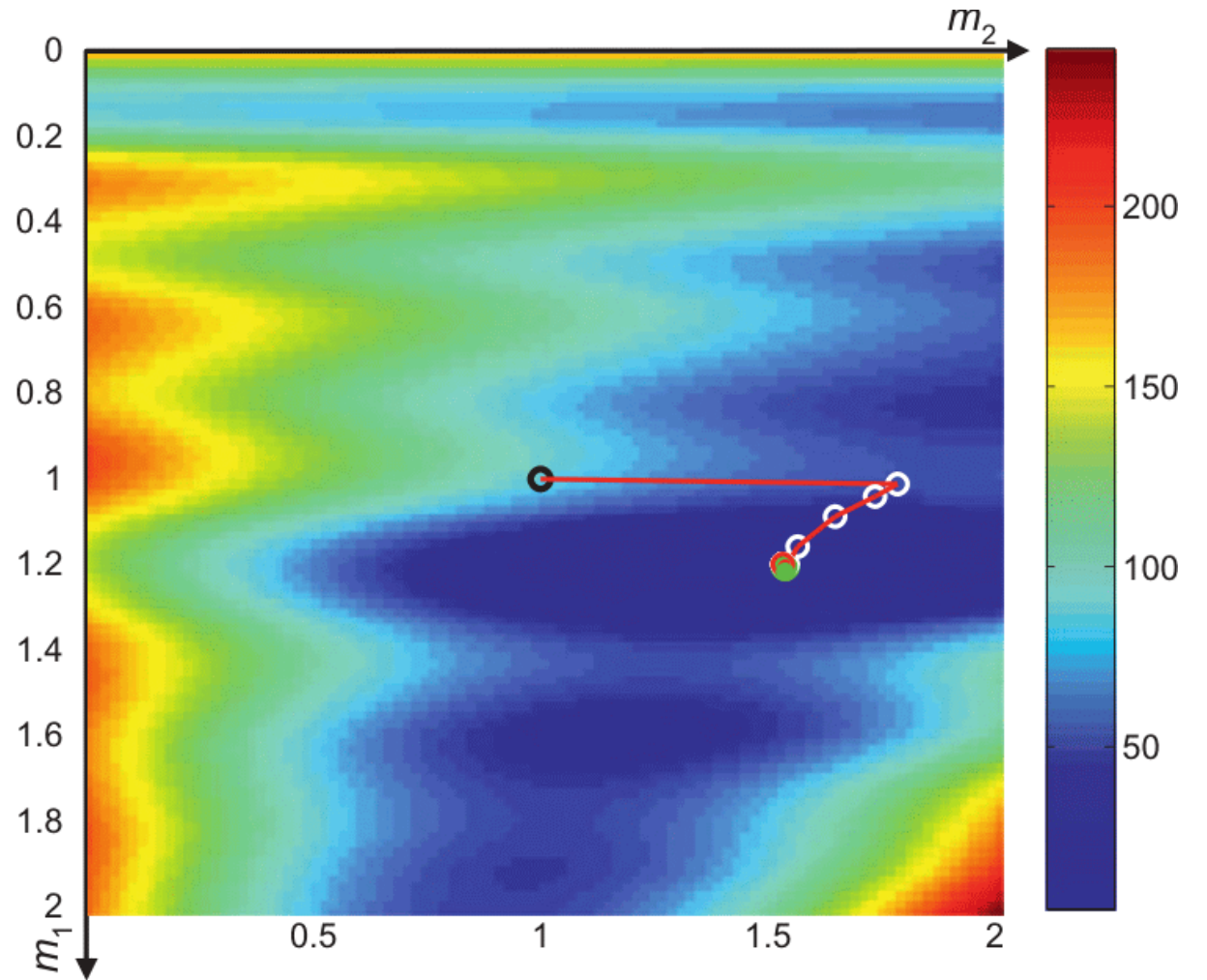
$$\Delta \mathbf{m} = (\mathbf{S}^T \mathbf{S} + \lambda \mathbf{C}^T \mathbf{C})^{-1} (\mathbf{S}^T (\mathbf{d} - \mathbf{f}(\mathbf{m})) + \lambda \mathbf{C}^T (\mathbf{c} - \mathbf{C} \mathbf{m}))$$

4. Optimize step length τ^n & update model: $\mathbf{m}^{n+1} = \mathbf{m}^n + \tau^n \Delta \mathbf{m}^n$
5. Compute model response & check convergence: proceed with 2.
 - maximum iteration number, decrease of Φ , reaching target $\chi^2 = 1$

Gradient vs. Newton (Menke)



Gradient method



Newtons method

Local vs. global regularization

Penalizing size or smoothness of

- the model update $\Delta \mathbf{m}$ (local) or
- the model $\mathbf{m}$ (global)

Levenberg-Marquardt algorithm

local damping of $\Delta \mathbf{m}$ (for stabilizing only)

$$\Delta \mathbf{m} = (\mathbf{J}^T \mathbf{J} + \lambda \mathbf{I})^{-1} \mathbf{J}^T (\mathbf{d} - \mathbf{f}(\mathbf{m}))$$

typically used for independent parameters:

- coefficients of fitting (e.g. petrophysical) models
- physical parameters & layer thicknesses of (few) layered model
- often used with logarithmic parameters preventing negatives
- λ usually decreased, i.e. $\lambda^k = \lambda^0 v^k$ (e.g., $v=0.5$ etc.)

Occam (smoothness-constrained) inversion

global regularization of the smoothness $\|\mathbf{C}\mathbf{m}\|$

$$\Delta\mathbf{m} = (\mathbf{S}^T\mathbf{S} + \lambda\mathbf{C}^T\mathbf{C})^{-1}(\mathbf{S}^T(\mathbf{d} - \mathbf{f}(\mathbf{m})) - \lambda\mathbf{C}^T\mathbf{C}\mathbf{m})$$

used for pre-discretized models: 1D, 2D, 3D, 4D

- full application of Gauss-Newton minimization
- different types of smoothness: 1st/2nd order, geostatistics
- decoupling different regions, region-wise regularization etc.
- regularization parameter λ often kept constant or slightly cooled

Resolution of non-linear inverse problems

- for linear problems directly using system matrix $\mathbf{G}$:

$$\mathbf{R}^M = \mathbf{G}^\dagger \mathbf{G}$$

$$\mathbf{R}^D = \mathbf{G} \mathbf{G}^\dagger$$

Resolution matrices for non-linear problems

$$\mathbf{m}^{k+1} = \mathbf{m}^k + \mathbf{S}^\dagger(\mathbf{d} - \mathbf{f}(\mathbf{m})) + \mathbf{C}^\dagger\mathbf{C}(\mathbf{m}^0 - \mathbf{m}^k)$$

with

$$\mathbf{S}^\dagger = (\mathbf{S}^T\mathbf{S} + \lambda\mathbf{C}^T\mathbf{C})^{-1}\mathbf{S}^T$$

$$\mathbf{C}^\dagger = (\mathbf{S}^T\mathbf{S} + \lambda\mathbf{C}^T\mathbf{C})^{-1}\mathbf{C}^T$$

and their combination $\mathbf{S}^\dagger\mathbf{S} + \mathbf{C}^\dagger\mathbf{C} = \mathbf{I}$

Model resolution for non-linear problems

$$\mathbf{m}^{k+1} = \mathbf{m}^k + \mathbf{S}^\dagger(\mathbf{d} - \mathbf{f}(\mathbf{m})) + \mathbf{C}^\dagger\mathbf{C}(\mathbf{m}^0 - \mathbf{m}^k)$$

assuming we “come close”, i.e. $\mathbf{m}^{est} = \mathbf{m}^{k+1}$ and $\Delta\mathbf{m}^k$ is small. With

$$\mathbf{d} = \mathbf{f}(\mathbf{m}^{true}) + \mathbf{n} = \mathbf{f}(\mathbf{m}^k) + \mathbf{S}(\mathbf{m}^{true} - \mathbf{m}^k) + \mathbf{n}$$

$$\Rightarrow \mathbf{m}^{est} = \mathbf{m}^k + \mathbf{S}^\dagger\mathbf{S}(\mathbf{m}^{true} - \mathbf{m}^k) + \mathbf{C}^\dagger\mathbf{C}(\mathbf{m}^0 - \mathbf{m}^k) + \mathbf{S}^\dagger\mathbf{n}$$

$$\Rightarrow \mathbf{m}^{est} = \mathbf{m}^k + \mathbf{S}^\dagger\mathbf{S}\mathbf{m}^{true} - (\mathbf{S}^\dagger\mathbf{S} + \mathbf{C}^\dagger\mathbf{C})\mathbf{m}^k + \mathbf{C}^\dagger\mathbf{C}\mathbf{m}^0 + \mathbf{S}^\dagger\mathbf{n}$$

$$\Rightarrow \mathbf{m}^{est} = \mathbf{R}^M\mathbf{m}^{true} + (\mathbf{I} - \mathbf{R}^M)\mathbf{m}^0 + \mathbf{S}^\dagger\mathbf{n}$$

Data resolution for non-linear problems

$$\mathbf{f}(\mathbf{m}^{k+1}) = \mathbf{S}(\mathbf{S}^\dagger(\mathbf{f}(\mathbf{m}^{true}) + \mathbf{d} - \mathbf{f}(\mathbf{m}^k))) + \mathbf{C}^\dagger \mathbf{C}(\mathbf{m}^0 - \mathbf{m}^k)$$

$$\mathbf{f}(\mathbf{m}^{est}) = \mathbf{R}^D \mathbf{f}(\mathbf{m}^{true}) + \mathbf{R}^D \mathbf{n} + (\mathbf{I} - \mathbf{R}^D) \mathbf{f}(\mathbf{m}^0)$$

$$\mathbf{f}(\mathbf{m}^{est}) = \mathbf{R}^D (\mathbf{f}(\mathbf{m}^{true}) + \mathbf{n}) + (\mathbf{I} - \mathbf{R}^D) \mathbf{f}(\mathbf{m}^0)$$

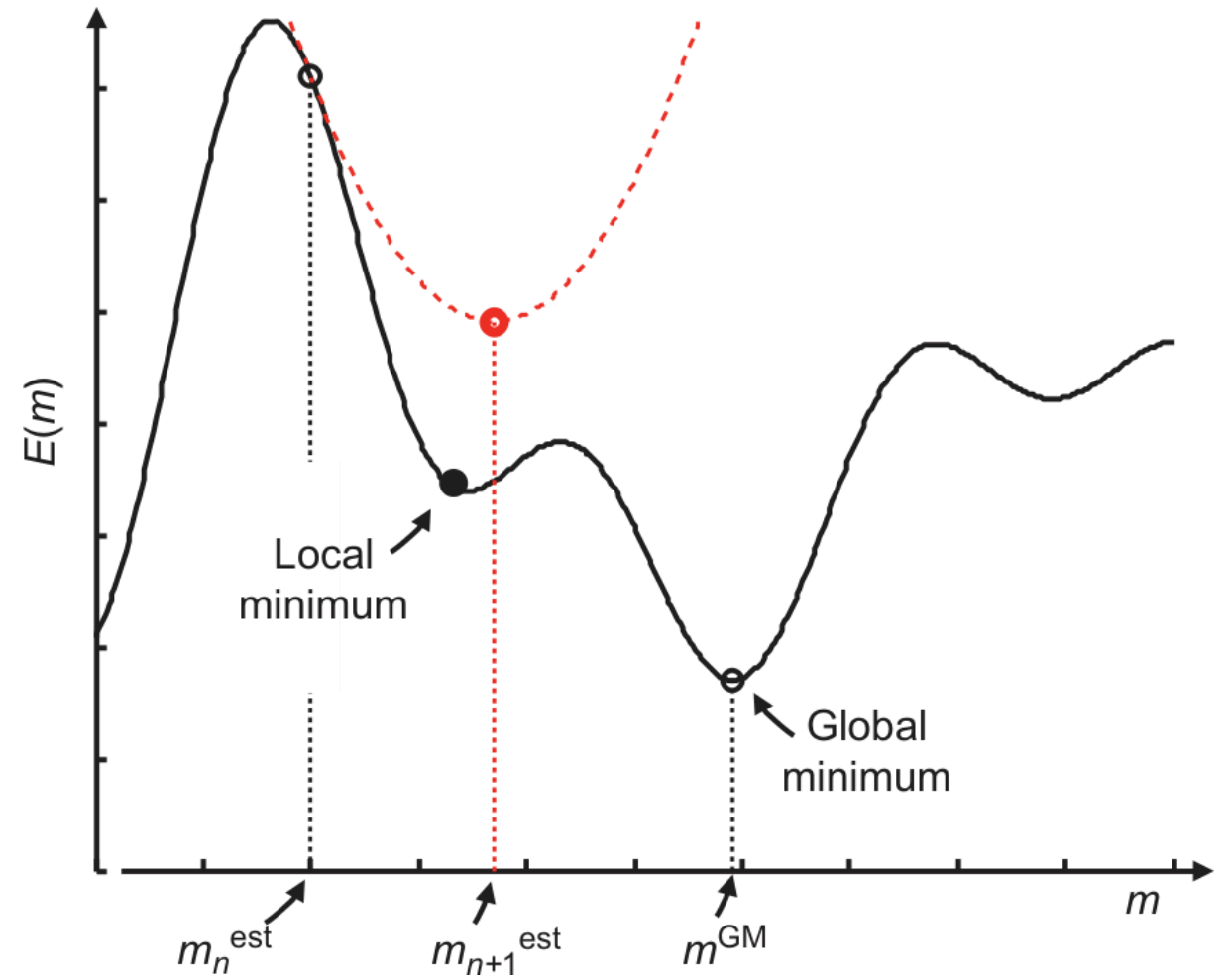
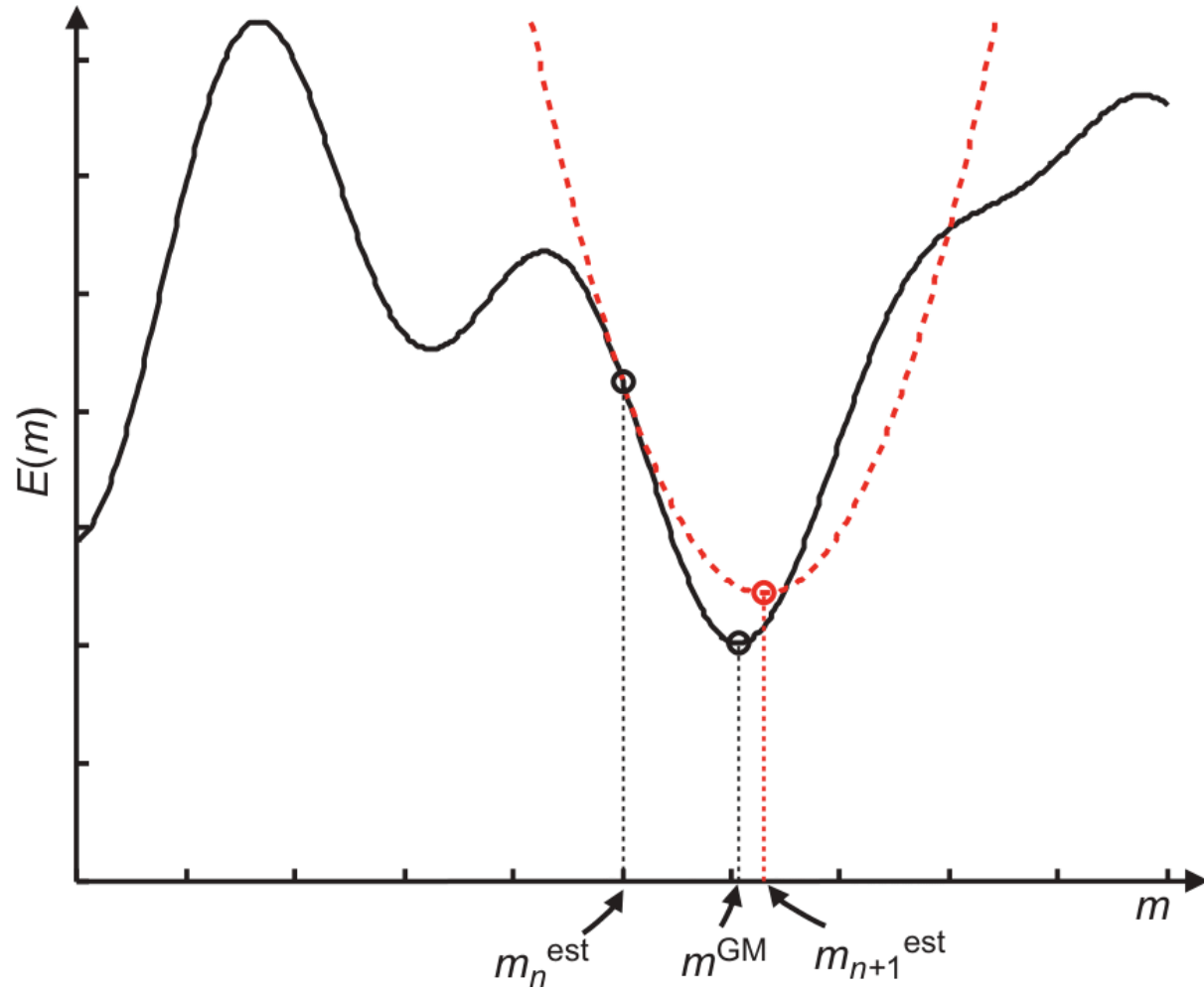
$$\mathbf{f}(\mathbf{m}^{est}) = \mathbf{R}^D \mathbf{d} + (\mathbf{I} - \mathbf{R}^D) \mathbf{f}(\mathbf{m}^0)$$

Response through data space & starting response through null space

Model resolution for non-linear problems

- resolution matrices $\mathbf{R}^M$ and $\mathbf{R}^D$ formed just like in linear inversion but with $\mathbf{S}^k$
- as $\mathbf{S}^k$ depends on $\mathbf{m}^k$ (near true), resolution is model-dependent
- all derived properties as before

Newton's method (Menke, 2012)



linearize with value, slope and curvature of Φ_d

Alternatives to grid search

Monte Carlo search

draw random samples and accept them if the error is improved

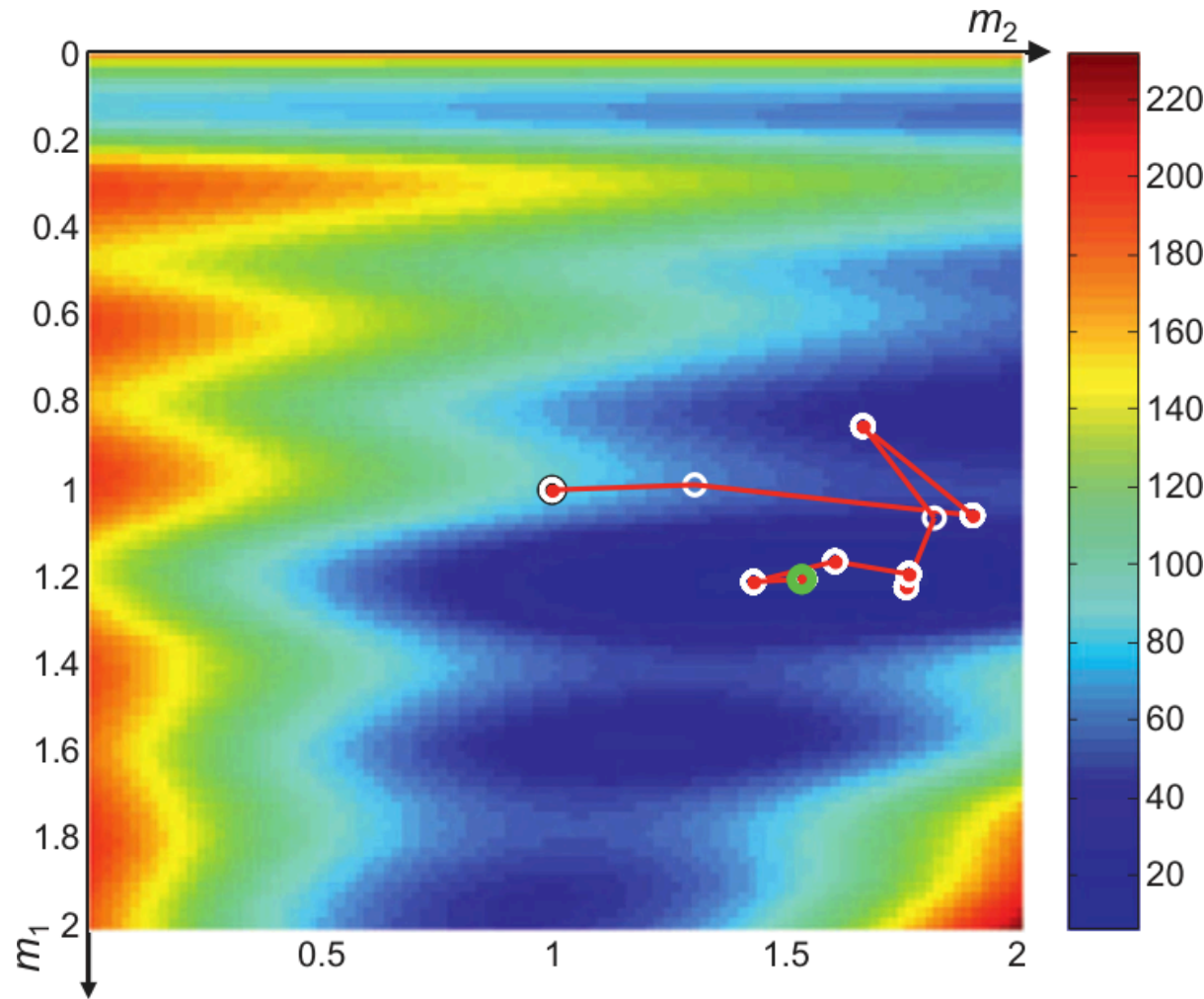
undirected search (Newtons method is directed)

Simulated annealing

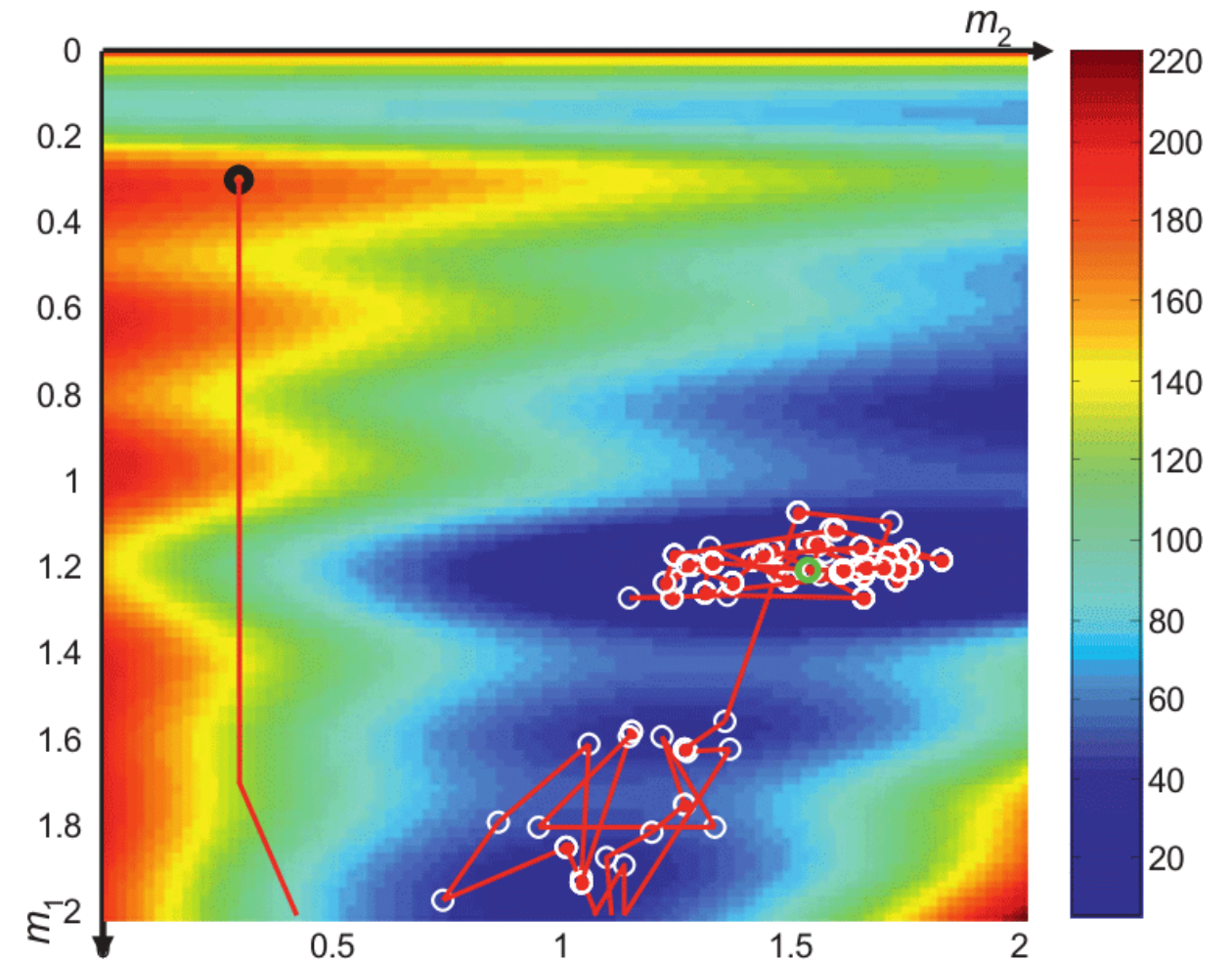
decrease temperature controlling particle movements:

high T : undirected, low T : search in vicinity of current model

Monte Carlo vs. Simulated Annealing



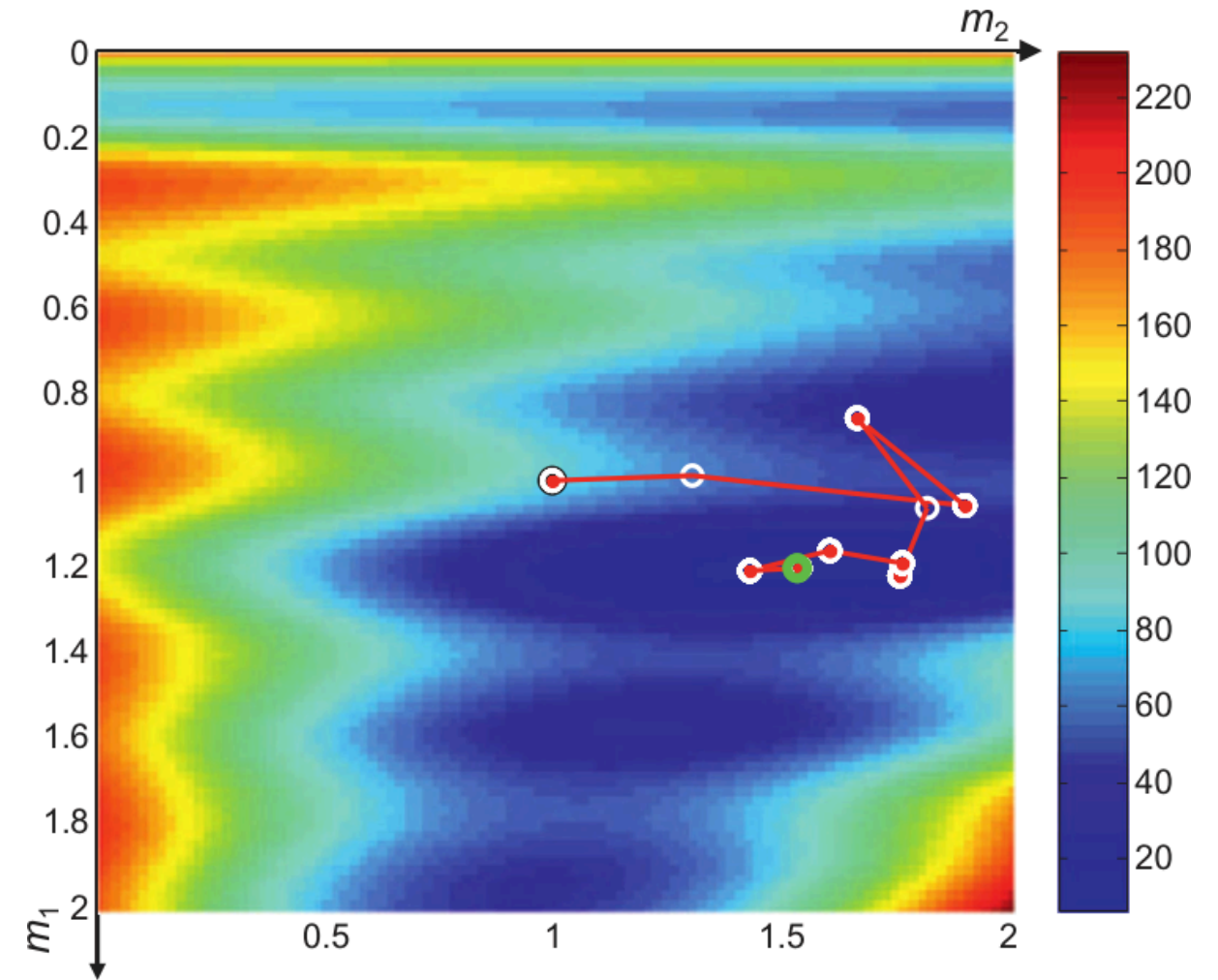
Monte Carlo method



Simulated Annealing

Monte Carlo methods

- Monte Carlos search: randomly draw solutions from grid
- accept solution only if better than old
- Markow-Chain-Monte-Carlo
- Metropolis-Hastings (Metropolis et al., 1953; Hastings, 1970)
- acceptance with some probability
⇒ need to look into that



Monte Carlo method